

2017

Instituto Politécnico da
Guarda

António Martins

SISTEMAS DIGITAIS I

Bases de numeração, códigos, e álgebra de Boole

Capítulo 1 - Sistemas de numeração

Introdução

Neste capítulo são apresentadas formas de representar números naturais, fraccionários e inteiros noutras bases de numeração, sendo apresentados os algoritmos de transição e as regras de representação de outras bases de numeração. Finaliza-se o capítulo com uma apresentação dos códigos mais comuns.

1.1. Bases de numeração de números naturais

Todos nós conhecemos a numeração actual e também a romana, que nos lembra que havia outra maneira de representar números. Em ambos os casos se usam símbolos cujo valor depende da posição. Assim 'IX' representa o número nove, enquanto se trocarmos os símbolos se obtém 'XI' que é o atual onze. A vantagem da numeração indo-árabe sobre a romana é o uso do zero, entre outras, o que permite que cinquenta, quinhentos ou cinco mil sejam escritos com apenas dois símbolos, sendo um repetido as vezes necessárias. Outra vantagem é na algoritmia, onde por exemplo na soma se alinham unidades sobre unidades, dezenas sobre dezenas, centenas sobre centenas etc. somando separadamente as unidades, as dezenas, as centenas etc. Quando se escreve o número 365 referimos um valor que contém três centenas, seis dezenas e cinco unidades. Usando a potência da base dez para a centena, que é a dezena da dezena tem-se:

$$365=300+60+5 \qquad =3 \times 10^2 + 6 \times 10^1 + 5$$

Na realidade até cinco se pode escrever usando a potência zero da base dez.

Este sistema é dito de base dez, já que usa dez símbolos, de 0 a 9, mas também porque esgotados os símbolos para as unidades se introduz à esquerda um novo algarismo, das dezenas, que ao fim de dez dezenas se introduz novo algarismo á esquerda, das centenas, e assim sucessivamente, com o milhar, etc. A associação ascendente em grupos de dez é assim fulcral neste sistema de numeração.

Outros povos desenvolveram outras bases para representar os números. Os babilónios desenvolveram a base 60 [1], e ainda existem reminiscências desse sistema na medida do tempo e dos ângulos. Como o tempo era medido pelo deslocamento angular da sombra de uma vara no solo, medir o tempo resumia-se a medir o ângulo, pelo que o sistema 60, ou sexagesimal, era usado para as duas grandezas. Assim a hora tem 60 minutos e este tem sessenta segundos, tal como o grau se divide em sessenta minutos e cada um destes em sessenta segundos. Os Yukis, uma tribo californiana [1], contava pelo espaço entre os dedos, pelo que usavam a base oito, hoje denominada octal. Podemos estudar esta base, já que é próxima da base dez e representar os números octais com os símbolos indo-árabes. Note-se

Bases de numeração, códigos e Álgebra de Boole

que o maior símbolo é sete, a que se segue o ‘dez’, já que em octal a ‘dezena’ será oito, cujo símbolo não existe. O ‘nove’ da base dez, cujo símbolo também não é usado em octal, será onze e assim sucessivamente. Ilustra-se a representação em octal na tabela seguinte.

Tabela 1.1 - Números na base oito

Base 10	0	1	2	3	4	5	6	7	8	9	10
Base 8	0	1	2	3	4	5	6	7	10	11	12

Nota: os números de base dez continuam-se a escrever normalmente. Números noutras bases usam a informação da base em índice.

Com dois algarismos o número mais alto é 77_8 a que se segue o 100_8 e assim sucessivamente. Tal como na base dez, os números em octal podem ser representados por um polinómio nas potências da base. Assim a ‘dúzia’ em octal equivale a:

$$12_8 = 1 \times 8^1 + 2 \times 8^0 = 8 + 2 = 10$$

Este resultado pode facilmente ser verificável na tabela 1.1.

Apresenta-se outro exemplo:

$$375_8 = 3 \times 8^2 + 7 \times 8 + 5 = 253$$

Pode agora apresentar-se o teorema fundamental da numeração escrita em qualquer base ‘r’. Assim tem-se [2]:

$$\overline{a_n a_{n-1} \dots a_1 a_0}_r = a_n \cdot r^n + a_{n-1} \cdot r^{n-1} + \dots + a_1 \cdot r + a_0$$

A barra indica que os símbolos debaixo dela não estão a multiplicar, mas que o seu conjunto forma um número qualquer na base ‘r’. Este teorema permite representar na base dez qualquer outro número apresentado noutra base.

Para saber como se passa um número da base dez para octal, por exemplo, suponha-se que se quer converter o número 2011 da base dez para octal sendo este representável, por hipótese, por quatro algarismos que se pretende descobrir. Assim tem-se:

$$2011 = \overline{a_3 a_2 a_1 a_0}_8$$

Na forma de potências, e usando o teorema fundamental da numeração, tem-se:

$$2011 = a_3 \cdot 8^3 + a_2 \cdot 8^2 + a_1 \cdot 8 + a_0$$

Colocando 8 em evidência nos primeiros três monómios do segundo membro tem-se:

$$2011 = (a_3 \cdot 8^2 + a_2 \cdot 8^1 + a_1) \cdot 8 + a_0$$

O que sugere que a_0 é o resto da divisão inteira de 2011 por 8, sendo o quociente dado pela expressão dentro dos parêntesis. Seja Q_1 o referido quociente. Evidenciando de novo 8 nas duas primeiras parcelas de Q_1 tem-se:

$$Q_1 = (a_3 \cdot 8^1 + a_2) \cdot 8 + a_1$$

Assim, a_1 é o resto da divisão de Q_1 por 8, sendo a expressão entre parêntesis o segundo quociente Q_2 , obtendo-se:

$$Q_2 = a_3 \cdot 8 + a_2$$

Ora a_2 é o resto da divisão de Q_2 por 8 e a_3 o quociente da mesma. Este algoritmo faz-se em sequência e chama-se algoritmo das divisões sucessivas. Apresenta-se o cálculo a seguir na figura 1.1:

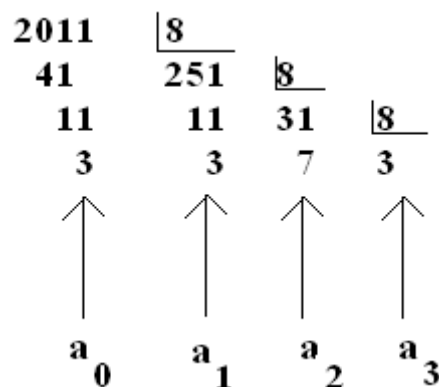


Figura 1.1 - Algoritmo de divisões sucessivas

O resultado é $2011 = 3733_8$.

Os métodos apresentados são generalizáveis a outras bases. Podem-se também efectuar operações na base oito desde que se elabore uma tabuada da referida base. Apresenta-se na tabela 1.2 a tabuada de adição da base oito.

Tabela 1.2 - Tabuada em octal

+	1	2	3	4	5	6	7
1	2	3	4	5	6	7	10
2	3	4	5	6	7	10	11
3	4	5	6	7	10	11	12
4	5	6	7	10	11	12	13
5	6	7	10	11	12	13	14
6	7	10	11	12	13	14	15
7	10	11	12	13	14	15	16

Bases de numeração, códigos e Álgebra de Boole

O zero continua a ser o elemento neutro da adição pelo que não foi considerado.

Tabela 1.3 -Tabuada de multiplicação

x	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7
2	2	4	6	10	12	14	16
3	3	6	11	14	17	22	25
4	4	10	14	20	24	30	34
5	5	12	17	24	31	36	43
6	6	14	22	30	36	44	52
7	7	16	25	34	43	52	61

Qualquer linha pode ser obtida por soma sucessiva de 4 a partir do primeiro algarismo. Por exemplo a linha 4 começa com $4 \times 1 = 4$ e depois o 4×2 pode obter-se somando 4, que daria 8, ou seja 10 em octal. $10 + 4 = 14$ e assim sucessivamente. Apresenta-se um exemplo de cálculo na base oito:

$$\begin{array}{r} 343 \\ \times 72 \\ \hline 706 \\ 3065 \\ \hline 31556 \end{array}$$

Figura 1.2 - Multiplicação em octal

Convida-se o estudante a converter os factores e o resultado para a base dez e verificar o cálculo na referida base.

1.1.1. A base dois ou binário.

Em binário só há dois símbolos, zero e um. A seguir ao um virá o 10, a ‘dezena’ binária que vale dois. Como depois de 10 vem 11 este representa três. Tal como na base dez depois de 99, que são os últimos algarismos, vem 100, também em binário depois de 11 se obtém o 100 que representa o número 4. Apresenta-se uma tabela com a conversão até 15.

Tabela 1.4 Conversão decimal-binário.

Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Binário	0	1	10	11	100	101	110	111	1000	1001	1010	1011	1100	1101	1110	1111

A mudança de base entre binário e decimal fazem-se com os mesmos métodos já referidos para octal. Assim a penúltima linha obtém-se, usando o teorema fundamental da seguinte forma:

$$1110_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2 + 0$$

$$= 8 + 4 + 2 = 14$$

Bases de numeração, códigos e Álgebra de Boole

Note-se que como os algarismos usados são apenas o 0 ou o 1, um método mais expedito consiste em apenas usar as potências de dois, associadas a cada um, obtendo-se imediatamente a segunda linha do cálculo. A conversão decimal para binário é feita por divisões sucessivas do número que se pretende escrever em binário por dois. Apresenta-se a conversão de 100 para binário.

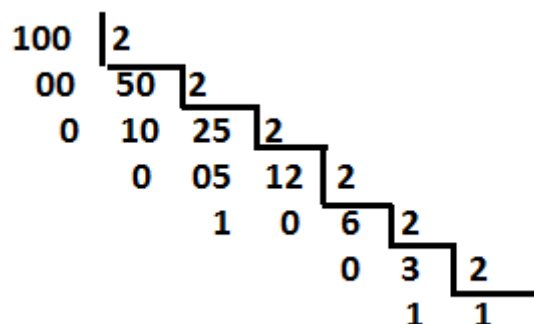


Figura 1.3 - Conversão de 100 para binário

O resultado é 1100100_2 .

As tabuadas em binário são quase iguais às da base dez. A tabuada de multiplicação é igual mas a de adição apresenta um resultado diferente para $1+1$ que é 10_2 (dois) em binário. Apresentam-se a seguir exemplos das quatro operações fundamentais:

$\begin{array}{r} \textcolor{green}{1} \\ 1101 \\ + 110 \\ \hline 10011 \end{array}$	$\begin{array}{r} 10111 \\ \times 11001 \\ \hline \textcolor{green}{1}10111 \\ \textcolor{green}{1}00000 \\ \textcolor{green}{1}00000 \\ \textcolor{green}{1}10111 \\ 10111 \\ \hline 100011111 \end{array}$	$\begin{array}{r} 1011101 \\ - 101 \\ \hline 0110 \\ - 101 \\ \hline 011 \end{array}$	$\begin{array}{r} 101 \\ \hline 10010 \end{array}$
$\begin{array}{r} 1101 \\ - 110 \\ \hline 0111 \\ \textcolor{green}{11} \end{array}$			

Figura 1.4

Os transportes aditivos, na adição e na multiplicação, estão representados a verde por cima da coluna. Os transportes para o subtrativo estão a verde por baixo. Na multiplicação as linhas de zeros não se costumam usar, avançando a linha seguinte duas casas.

1.1.2. Base hexadecimal

Uma base maior do que dez tem mais de dez símbolos elementares. Nestes casos depois do último símbolo numérico usam-se as maiúsculas do alfabeto latino. Na base dezasseis, a mais

Bases de numeração, códigos e Álgebra de Boole

importante destas, precisam-se 16 símbolos, sendo dez numéricos e os restantes seis as letras A, B, C, D, E e F [2]. Apresenta-se na tabela 1.5 as equivalências numéricas até 15.

Tabela 1.5 Conversão decimal-hexadecimal.

Decimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Binário	0	1	10	11	100	101	110	111	1000	1001	1010	1011	1100	1101	1110	1111

O número 16 será escrito como 10 e os seguintes obtêm-se incrementando o algarismo das unidades até 1F que precede o 20, e assim sucessivamente. Apresenta-se um exemplo de conversão do número decimal 45 para hexadecimal. O resultado da conversão é $2D_{16}$.

$$\begin{array}{r} 45 \quad \underline{16} \\ 13 \quad 2 \end{array}$$

Figura 1.5

1.1.3. Conversão de números entre bases que são potências de dois.

A representação em binário, usada pelos computadores, usa sequências de algarismos longas. A conversão direta para octal ou hexadecimal, bases potências de dois, resolve esse problema. Considere-se o seguinte número binário e a sua representação em potências da base:

$$\begin{aligned} 101001110101_2 \\ = 1 \times 2^{11} + 0 \times 2^{10} + 1 \times 2^9 + 0 \times 2^8 + 0 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 \\ + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \end{aligned}$$

Pretende-se converter o número para hexadecimal. Como esta base é a quarta potência de dois, agrupam-se os termos em grupos de 4 partindo dos menos significativos.

$$\begin{aligned} 101001110101_2 \\ = (1 \times 2^{11} + 0 \times 2^{10} + 1 \times 2^9 + 0 \times 2^8) \\ + (0 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4) \\ + (0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0) \end{aligned}$$

Em cada parêntesis evidencia-se a potência de dois que permite manter os pesos 8421 naqueles.

$$\begin{aligned} 101001110101_2 \\ = (1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0) \times 2^8 \\ + (0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0) \times 2^4 \\ + (0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0) \end{aligned}$$

Como $2^8=16^2$ tem-se:

$$101001110101_2 = 10 \times 16^2 + 7 \times 16^1 + 5 \times 16^0$$

Finalmente:

$$101001110101_2 = A75_{16} \quad (1010)(0111)(0101)_2$$

Se fizéssemos grupos de três teríamos o número vertido para octal. Assim:

$$101001110101_2 = 5165_8 \quad (101)(001)(110)(101)$$

Este método permite uma representação compacta da informação em base dois com poucos símbolos, usando-se normalmente o hexadecimal.

1.2. Os números fracionários noutras bases

Uma fracção diz-se própria se for menor do que um. Como os símbolos zero e um são os mesmos em todas as bases, então uma dízima representativa de uma fracção própria continuará a representar uma fracção própria mesmo se mudarmos de base de numeração. Quando referimos o número 0,728 pensamos num número constituído por sete décimas, duas centésimas e oito milésimas. Representando a referida dízima em potências da base dez tem-se:

$$0,728 = 7 \times 10^{-1} + 2 \times 10^{-2} + 8 \times 10^{-3}$$

Ora esta expressão numérica é o teorema fundamental da numeração escrita, generalizado para as potências negativas, associadas aos números à direita da vírgula. Assim qualquer dízima representativa de uma fracção própria na base 'r' pode ser calculada na base dez usando a seguinte expressão [2]:

$$\overline{0, a_{-1} a_{-2} \dots a_{-n}_r} = a_{-1} \cdot r^{-1} + a_{-2} \cdot r^{-2} + \dots + a_{-n} \cdot r^{-n}$$

Por exemplo 0,4 em octal representa:

$$0,4_8 = 4 \cdot 8^{-1} = \frac{4}{8} = 0,5$$

Contudo 0,13 representa uma dízima infinita na base dez. Ou seja uma representação finita numa base pode ser infinita noutra. Quantos algarismos se devem preservar em casos destes? A regra a reter é que a precisão com que o número é representado não pode aumentar com a operação de mudança de base [2]. No primeiro exemplo 0,728 está representado com uma precisão de 1 em 103 visto que um número com três casas decimais pode representar 1000 quantidades diferentes desde 0,000 até 0,999.

A precisão de um número com 'n' casas à direita da vírgula, na base 'r', é de [2]:

$$1 \text{ em } 'r^n'.$$

Bases de numeração, códigos e Álgebra de Boole

No segundo caso $0,4_8$ a precisão é de 1 em 8 e no terceiro caso $0,1_3$ a precisão é de 1 em 3. Nestes casos não faz sentido representar os números na base dez com uma casa decimal já que isso aumentava a precisão do número. O resultado deverá ser arredondado a 1 na conversão octal e a 0 na conversão do ternário, já que o resultado seria 0,3. Apresenta-se um exemplo de conversão de hexadecimal para decimal.

$$0,C3_{16} = 12 \times 16^{-1} + 3 \times 16^{-2}$$

A precisão do número é de 1 em 16^2 , ou 1 em 256. O uso de duas casas decimais na base dez dá uma precisão de 1 em 100 pelo que é suficiente. Assim o resultado é 0,76. (Não se esqueça de arredondar!)

A questão que falta resolver é como representar noutras bases uma dízima conhecida na base dez. Pretende-se representar 0,62 em octal. Como a precisão é de 1 em 100, duas casas á direita da vírgula em octal são suficientes já que têm uma precisão de 1 em 64.

O algoritmo é o seguinte:

$0,62 \times 8 = 4,96$ extrai-se o 4, primeiro algoritmo á direita da vírgula.

$0,96 \times 8 = 7,68$ extrai-se 0 7, segundo algoritmo á direita da vírgula.

$0,68 \times 8 = 5,44$ O terceiro algoritmo seria 5. Neste caso não é usado mas indica que o arredondamento é para cima.

$$0,62 = 0,475_8 = 0,50_8$$

Apresenta-se a justificação teórica para o algoritmo exposto, chamado de multiplicações sucessivas. Assim, 0,62 deverá ter dois símbolos na base de chegada, pelo que se pode escrever:

$$0,62 = \overline{0,ab}_8 = a \times 8^{-1} + b \times 8^{-2}$$

Multiplicando por 8 tem-se:

$$8 \times 0,62 = a \times 8^0 + b \times 8^{-1}$$

Ou:

$$4,96 = a + b \times 8^{-1}$$

Resulta que $a=4$, já que as partes inteiras têm que ser iguais tais como as partes fraccionárias. A equação reduz-se a:

$$0,98 = b \times 8^{-1}$$

Multiplicando de novo por 8 tem-se:

$$7,68 = b$$

Assim tem-se $b=7$. Um eventual terceiro algarismo obter-se-ia multiplicando por 8 a dízima restante, ou seja 0,68 o que daria 5,44. O resultado seria:

$$0,62 = 0,475_8 = 0,50_8$$

Quando se calculou ‘ b ’ podia-se logo arredondar este ao número seguinte. Note que 0,58 está mais perto da unidade que o ‘nosso’ 0,5. O resultado deve ser apresentado com as duas casas à direita da vírgula.

O método apresentado chama-se das multiplicações sucessivas [2].

Exemplo: Converta 0,1 para binário.

Resolução - O número tem uma precisão de 1 em 10 pelo que terá 3 símbolos à direita da vírgula em binário. As multiplicações sucessivas apresentam-se na figura seguinte:

0,1	0,2	0,4	0,8
$\times 2$	$\times 2$	$\times 2$	$\times 2$
0,2	0,4	0,8	1,6
$\Downarrow$	$\Downarrow$	$\Downarrow$	$\Downarrow$
0	0	0	1

Figura 1.6 Algoritmo de multiplicações sucessivas

Se o aluno fizer apenas três multiplicações obtém $0,000_2$. Este valor tem um erro de uma décima para 0,1. Fazendo a quarta multiplicação e extraíndo o 1, este valor mostra que o arredondamento deve ser para o valor superior, ou seja $0,001_2$. De facto este número representa $1/8$, ou seja 0,125, valor este que tem um erro de 0,025, ou seja, quatro vezes menor que o erro anterior de 0,1.

Exemplo: Converter para binário $0,F3_{16}$. A precisão é de 1 em 256. Se convertermos directamente temos $0,F3_{16}=0,11110011_2$. A precisão é de 1 em 2^8 , ou seja 1 em 256. É fácil verificar que se trata do mesmo número. A conversão direta de hexadecimal para binário não requer um cálculo de precisão. Atenção que a conversão de binário para hexadecimal não goza da mesma propriedade, a não ser que o número de bits seja múltiplo de 4.

Bases de numeração, códigos e Álgebra de Boole

Exercícios

1 - Escreva os números decimais de 0 a 16 nas bases seguintes: binário, octal, duodecimal e hexadecimal.

2 - Converta para a base dez: 1010_2 4535_6 $3FD_{16}$ 135_8

3 - Converter para hexadecimal: 135 3572_8 $A5B7_{12}$

4 - Converter para binário: $3F52_{16}$ 147_8 $EA5B_{16}$ 4052_8

5 - Realize as seguintes operações em binário, sabendo que todos os números estão representados na base dois.

$101111+101110$ $101101+10011$ $1110-111$

1011×1100 $1011101/101$

6 - Verifique o resultado da última operação efectuando a operação inversa

7 - Converta para a base dez a seguinte quantidade octal $352,1612$. Calcule o número de casas decimais na base de chegada. Repita para $A1C, B5_{16}$ e $574^a, B21_{12}$

8 - Converta para octal o seguinte número decimal $135,325$. Calcule o número de casas decimais na base de chegada.

9 - Efectue em octal $42 : 34$. Sugestão: Elabore as tabuadas octais antes.

10 - Converta para binário $35,271$

11 - Converta $2121,1023$ para binário

12 - Calcule em octal $725,3_8 + 101011,101_2$

1.3. A representação de números inteiros.

A representação clássica de números reais é o módulo do número precedido do seu sinal. A primeira questão que se deve colocar tem a ver com a representação do sinal. A norma a usar é considerar o zero como sinal '+' já que é costume omitir o sinal dos números positivos, tal como os zeros à esquerda. Assim num registo decimal de três algarismos, o maior número positivo representável será 099. Para representar o sinal '-', utiliza-se o maior algarismo da base, ou seja, o 9 [2]. Assim, o menor número representável no mesmo registo será 999.

Exemplo: a adição algébrica seguinte $(-45)+(-28)$, será representada num registo de três algarismo: 945+928, não sendo preciso os parêntesis. O resultado será 973.

A adição algébrica, quando os números estão representados por sinal e módulo, é complexa para automatizar. Resumindo-a:

- Se as duas parcelas têm o mesmo sinal, o resultado tem o mesmo sinal, tendo como módulo a soma dos módulos das parcelas.
- Se as duas parcelas têm sinais diferentes, o resultado tem o sinal da parcela de maior módulo, sendo o módulo do resultado a diferença entre o módulo da maior parcela e o módulo da menor.

Este algoritmo requer um comparador de sinal, um somador, um comparador de módulos e um subtrator. É muita maquinaria!

1.3.1. Representação de inteiros em complemento para a base diminuída.

A alternativa a esta representação de números reais é a representação em complemento que facilita imenso a soma algébrica. Apresentar-se-á a representação de números na base dez, não só por motivos pedagógicos, já que é mais fácil de entender, mas também porque é usada em códigos por empresas de referência como a IBM [3]. A representação será depois generalizada a outras bases com particular interesse no binário.

Nesta representação os números positivos são representados por sinal, '0', e módulo, tal como na representação sinal e módulo. Os números negativos serão representados pelo sinal 'r-1', sendo 'r' a base de representação, seguido do complemento para 'r-1' do módulo do número.

O complemento para 'r-1' de um número N é definido por R. Sandige da forma seguinte [4]:

$$C(N) = r^i - r^{-f} - N$$

Em que 'r' é a base em que o número está representado, 'i' o número de algarismos inteiros, 'f' o número de algarismos fraccionários e 'N' o número cujo complemento se pretende calcular.

Vamos simplificar, considerando a base 10 num registo com sinal e três algarismos inteiros e nenhum fraccionário. Então na fórmula anterior tem-se 'r=10', 'i=3' e 'f=0' obtendo-se uma formulação mais simples:

$$C(N) = 10^3 - 10^0 - N = 999 - N$$

O complemento é assim a diferença para o maior número que se pode escrever no registo que é 0999. Note-se que o cálculo do complemento requer uma subtração que não gera transporte em nenhuma coluna.

Exemplo 1 - Representar 7, 456, -3, -29 e -801 em complemento para 9 num registo de sinal e 3 algarismos inteiros.

Solução: para os positivos obtém-se 0007 e 0456.

Para -3:

Sinal é 9. Complemento é 999-3, ou seja 996. A representação é 9996.

Bases de numeração, códigos e Álgebra de Boole

Para -29:

Sinal é 9. Complemento é 999-29, ou seja 970. A representação é 9970.

Para -801:

Sinal é 9. Complemento é 999-801, ou seja 198. A representação é 9198.

A grande vantagem deste método é que a soma algébrica é grandemente simplificada, reduzindo-se a somar os números, com o sinal incluído como se fosse mais um algarismo, tendo o cuidado de voltar a somar o transporte final ao resultado, ou seja se o transporte final for um deve incrementar-se uma unidade ao resultado obtido.

Exemplo 2 - Calcular 5-3 usando o mesmo registo do caso anterior:

Número na forma sinal e módulo	Número em complemento para 9	Soma em complemento
5	0005	0005
-3	9996	9996
		1 0001
		___1
		0002

Exemplo 3 - Calcular -3-29 usando o mesmo registo do caso anterior. A solução apresenta-se na figura 1.7, com o transporte final a ser de novo somado:

9996
9970

9966
1

9967

Figura 1.7 - Soma em complemento para 9

O resultado é negativo, já que começa por 9. O complemento do módulo é 967. Se 'N' for o módulo do número cujo complemento é 967 tem-se, de acordo com a fórmula do complemento:

$$967 = 999 - N$$

$$\text{Logo, } N = 999 - 967 = 32$$

O resultado representa o número -32. Fez-se de novo a subtracção para 999.

Um caso importante que importa considerar, é a ultrapassagem da capacidade do registo, o que no registo anterior acontece se o resultado da soma ultrapassar 999. Se o aluno somar 200 com 800, ou seja 0800+0200 obterá 1000. O símbolo '1' não é válido para sinal pelo que a

máquina detetará, deste modo, que houve ultrapassagem da capacidade do registo. O mesmo acontece para números negativos.

Exemplo 4 - Representar $1D3_{16}$ e $-DE_{16}$ na forma de complemento para 'r-1', ou seja, para 15 num registo de sinal e três algarismos.

Caso $1D3_{16} = 01D3_{16}$

Caso $-DE_{16}$: o sinal é 'F' e o complemento do módulo é $FFF_{16}-DE_{16}=F21_{16}$. Logo a representação é $FF2116$.

Exemplo 5. Com o registo do exemplo anterior efectuar $C_{16}-5_{16}$ em complemento para 15.

Solução: C_{16} é representável por $000C_{16}$. O complemento do módulo do subtrativo é $FFF_{16}-5_{16}=FFFA_{16}$. Logo o subtrativo é representável por $FFFA_{16}$.

$$\begin{array}{r} 000C \\ FFFA \\ \hline 0006 \\ 1 \\ \hline 0007 \end{array}$$

Figura 1.8 - soma em hexadecimal

Complemento para 1 em binário

Na base dois, os sinais '+' e '-' são representados por 0 e 1. A existência de apenas dois símbolos dificulta a deteção de ultrapassagem da capacidade de um registo já que qualquer símbolo é válido. Considere-se um caso simples de um registo de 4 bites com sinal incluído pelo que só terá três bites para o módulo do número, se este for positivo, ou para o respectivo complemento se for negativo. A fórmula do complemento fica com o seguinte aspeto, supondo que todos os números estão em binário:

$$C(N) = 10^3 - 10^0 - N = 111 - N$$

O complemento continua a ser a diferença para o maior número que o registo suporta.

Exemplo 6 - representar os números 6 e -2 em binário na forma de complemento.

No caso de 6, como o número é positivo a solução é sinal '0' e módulo '110', pelo que se representa na forma 0110_2 .

No caso de '-2', o sinal é negativo, logo é '1' seguido do complemento do módulo que é a diferença para 111_2 . Neste caso tem-se $C(2) = 111_2 - 10_2$ ou seja 101_2 . Então, a representação solicitada é 1101_2 .

Bases de numeração, códigos e Álgebra de Boole

Na tabela seguinte apresentam-se todas as possibilidades de representação para o registo considerado. Na primeira coluna apresenta-se o valor decimal, na segunda coluna a representação em sinal e módulo e na terceira coluna representam-se os números na forma de complemento para 'r-1' ou seja para um, já que 'r' é dois.

Tabela 1.6 - Representação de reais

Decimal	Binário normal	Binário em complemento
7	0111	0111
6	0110	0110
5	0101	0101
4	0100	0100
3	0011	0011
2	0010	0010
1	0001	0001
0	0000	0000 ou 1111
-1	1001	1110
-2	1010	1101
-3	1011	1100
-4	1100	1011
-5	1101	1010
-6	1110	1001
-7	1111	1000

Note-se que existem duas representações para o zero, sendo tal facto um sério defeito. Outra característica extremamente importante é a propriedade de que **o complemento se obtém por troca de símbolos, ou seja, o zero passa a um e vice-versa**, não havendo necessidade de um subtrator, como se pode verificar facilmente na tabela anterior.

Apresentam-se alguns exemplos simples:

5+2	5-2	2: 010	4	0100
			-6 - 110	1001
0101	0101	troca de símbolos	-2	1101
0010	1101			
0111	1 0010			simétrico é 0010
	1			por troca de
	0011			símbolos

Figura 1.9 - Operações em complemento para um

Como se verifica nunca se fazem subtrações, apenas somas.

Exercícios - Num registo de 8 bites um dos quais é de sinal, responda ou calcule:

Exercício 1 - Qual o maior número positivo representável?

Exercício 2 - Qual o menor número negativo que se pode guardar?

Exercício 3 - Qual o menor número em módulo?

Exercício 4 - Calcule $23+42$

Exercício 5 - Calcule $57-30$

Exercício 6 - Calcule $43-120$

Exercício 7 - Calcule $-18-79$

Exercício 8 - Calcule $95+42$

Exercício 9 - Calcule $-38 -117$

As soluções são facilmente obtidas fazendo o cálculo na base dez. Há dois casos em que a capacidade do registo é ultrapassada.

1.3.2. Representação de inteiros em complemento para a base.

Na representação em complemento para a base, complemento para dez em decimal e complemento para dois em binário, as regras de representação são as mesmas da subsecção anterior, ou seja os números positivos são representados por sinal, 0, e módulo, sendo os números negativos representados por sinal, 1, e o complemento do módulo. O cálculo do complemento é que é diferente, usando outra fórmula. Assim, em binário por exemplo tem-se: 'Complemento para dois de um número 'x' de 'n' bites é o resultado da operação $2^n - x$ '[2].

De outra forma, Sandige [4] refere que o complemento para a base se pode obter do complemento para a base diminuída (diminished radix complement) somando '1' ao bit menos significativo do número, ou seja, o complemento para dois pode obter-se por troca de símbolos e somando uma unidade ao número, mesmo que tenha componente fracionária. Note-se que neste código apenas existe uma representação para o zero, o que é uma vantagem. Na operação de soma o transporte final é agora desprezado, já que foi previamente adicionado ao complemento. Ilustra-se a teoria com alguns exemplos. Considera-se ainda um registo de oito bites, sinal incluído.

Exemplo 1: Efetuar $52-47$.

A solução apresenta-se de seguida. Ora $52=00\ 110\ 100_2$ e $47=00\ 101\ 111_2$. Trocando símbolos neste padrão de bites e somando 1 obtém-se $-47=11\ 010\ 001_2$. A adição apresenta-se na figura seguinte e o resultado é 5.

$$\begin{array}{r} 00\ 110\ 100 \\ +\ 11\ 010\ 001 \\ \hline 100\ 000\ 101 \end{array}$$

Figura 1.10 $52-47$ em complemento para a base

Exemplo 2 Efetuar 47-52.

Solução: Trocando os símbolos da representação de 52, obtida no exemplo anterior, e somando 1 tem-se $-52 = 11\ 001\ 100$. A operação apresenta-se na figura 1.11.

$$\begin{array}{r} 47\ 00\ 101\ 111 \\ -52\ 11\ 001\ 100 \\ \hline 11\ 111\ 011 \end{array}$$

Figura 1.11

O resultado é negativo e para o interpretar calcula-se o simétrico, ou seja, o complemento para a base. Assim, trocando símbolos e somando 1, obtém-se $00\ 000\ 101$. O resultado da figura 1.11 representa o número -5, como seria de esperar.

Problemas deste género podem ser treinados pelo estudante, usando a calculadora do Windows em modo programador. Se começar por executar a operação 0-1, em binário, vai obter a representação para a base do resultado, -1, podendo verificar o tamanho dos registos do PC.

As considerações sobre ultrapassagem da capacidade do registo, enunciadas na subsecção anterior, mantêm-se válidas.

Exemplo 3 - Num registo de 8 bites, sendo um de sinal, 4 bites para a parte inteira e 3 para a parte fracionária execute $3,1-2,7$.

Solução: A representação de 3,1 em binário é, no registo considerado, $0\ 0011,001_2$ e a representação de 2,7 é, no mesmo registo, $0\ 0010,110_2$. Trocando símbolos e somando 1 ao último bit obtém-se $-2,7 = 1\ 1101,010_2$. A operação ilustra-se na figura 1.12.

$$\begin{array}{r} -2,7\ 1\ 1101,010 \\ 3,1\ 0\ 0011,001 \\ \hline +0\ 0000,011 \end{array}$$

Figura 1.12 Execução da operação $3,1-2,7$

O resultado é $\frac{1}{4} + \frac{1}{8}$ valor próximo de 0,4 com um erro de 0,025. Experimente o leitor converter 0,4, resultado do exercício, para binário.

Exemplo 4 - Num registo decimal de sinal e 3 algarismos efectuar $728-359$, usando uma representação de números de complemento para a base, para dez neste caso.

Solução: O complemento para 9 de 359 é 640 pelo que somando 1 se obtém o complemento para dez que é 641. Assim, -359 é representável em complemento para a base na forma 9641. A solução com o respectivo resultado ilustra-se na figura 1.13.

$$\begin{array}{r} 0728 \\ +9641 \\ \hline 40369 \end{array}$$

Figura 1.13 Execução de 728-359

Exemplo 5 - Num registo hexadecimal de sinal e 3 algarismos efectuar $2A5_{16} - 13B_{16}$, usando uma representação de números de complemento para a base, para dezasseis neste caso.

Solução. O complemento para F de $13B_{16}$ é $EC4_{16}$ ($FFF - 13B$), pelo que somando 1 se obtém o complemento para a base, que é $EC5_{16}$. A representação do subtractivo é então $FEC5_{16}$, sendo o aditivo representado por $02A5_{16}$. A operação apresenta-se na figura 1.14.

$$\begin{array}{r} 02A5 \\ + FEC5 \\ \hline 4016A \end{array}$$

Figura 1.14 Execução de $2A5 - 13B$

1.4 Códigos

Pretende-se codificar ‘m’ mensagens usando um conjunto de ‘s’ símbolos distintos. O número de símbolos a usar chama-se valência do código [2]. Como exemplo de um código apresenta-se na Tabela 1.7 um dos mais antigos, o código de Morse [5]:

Tabela 1.7 Código de Morse

Letra	Código Internacional	Letra	Código Internacional
A	.-	N	-.
B	-...	O	---
C	-.-.	P	.-.
D	-..	Q	-.-.
E	.	R	.-.
F	..-.	S	...
G	--.	T	-
H		U	..-
I	..	V	...-
J	.-..	W	.-.
K	-. -	X	-..-
L	.-..	Y	-. -.
M	--	Z	-. .

Bases de numeração, códigos e Álgebra de Boole

Este código tem valência 2, já que apenas usa dois símbolos, ponto e traço. É um código que pode ter apenas um símbolo, para codificar a letra 'E' ou quatro para outras letras. O número de símbolos por palavra diz-se comprimento da palavra. Códigos que não têm o mesmo número de símbolos por mensagem dizem-se irregulares.

Um código diz-se regular se todas as mensagens codificadas tiverem comprimento fixo [2].

Um exemplo de um código regular é o ASCII (American Standard Code for Information Interchange). Como se pode verificar na Tabela 1.8 cada carater tem sete símbolos '0' ou '1'. Assim a codificação da letra 'A' é 100 0001. Os primeiros três símbolos são lidos em cima, conforme mostra o arco a vermelho, e os restantes quatro são lidos na coluna da esquerda conforme mostra o segmento de reta também a vermelho. O código ASCII diz-se alfanumérico já que codifica letras e números. Um código de valência dois diz-se binário.

Tabela 1.8 Código ASCII [2]

$b_6b_5b_4$	000	001	010	011	100	101	110	111
$b_3b_2b_1b_0$								
0000	NUL	DLE	SP	0	@	P	'	p
0001	STX	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Em Sistemas Digitais estamos particularmente interessados em código numéricos. Uma maneira de codificar os números decimais é usar a sua representação em binário natural que contudo geraria um código irregular. Para superar este defeito codificam-se só os algarismos de '0' a '9' colocando zeros à esquerda para se obter um código regular de comprimento 4. O resultado apresenta-se na tabela 1.9 e o código chama-se Binary Coded Decimal, BCD, ou Decimal Codificado em Binário. Assim em 93 é codificado separadamente o '9' e o '3' obtendo-se 1001 0011.

Tabela 1.9 Código BCD

Algarismo da base 10	Código Decimal codificado em Binário
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

O código BCD também se designa por código BCD 8421. Os algarismos significam a parcela a adicionar quando existe um '1' no lugar de algarismo. Assim 1000 têm um '1' no lugar do 8, logo a sequência representa o número 8, enquanto a sequência 0110 têm um '1' no lugar do 4 e outro '1' no lugar do 2, logo 0110 representa $4+2 = 6$. Note que há palavras que não pertencem ao código, como por exemplo 1111.

Um código numérico diz-se ponderado se a cada bit se faz corresponder um certo peso, bastando multiplicar cada bit pelo seu peso e somar os produtos para se obter o equivalente decimal da palavra do código [2].

Existem outros códigos numéricos que se apresentam na tabela 1.10.

Tabela 1.10 Códigos binários

N	BCD	BCD excesso 3	Aiken 2421	Johnson
0	0000	0011	0000	00000
1	0001	0100	0001	10000
2	0010	0101	0010	11000
3	0011	0110	0011	11100
4	0100	0111	1000	11110
5	0101	1000	1011	11111
6	0110	1001	1100	01111
7	0111	1010	1101	00111
8	1000	1011	1110	00011
9	1001	1100	1111	00001

O código BCD com excesso de três obtém-se somando três unidades ao algarismo que queremos codificar e convertendo depois para binário. Assim se quiser codificar o número 4,

acrescento 3 e codifico o resultado que é 7, ou seja 0111. Este código não é ponderado mas existe pelo menos um '1' em cada palavra o que permite detetar erros ou interrupções nas sequências. Repare-se que a partir do algarismo 'N' posso obter '9-N' por troca de zeros por uns e vice-versa.

Um código diz-se auto-complementar se para cada algarismo decimal 'N', o seu complemento para 9, resultante da subtracção 9-N, se obtém por troca de símbolos, zeros por uns e vice-versa. [2]

Verifica-se facilmente que o código de Aiken é simultaneamente ponderado e auto-complementar.

O código de Johnson á obtido por rotação para a direita de todos os bites. O bit da direita é substituído pelo complemento do bit mais à esquerda. Neste código de uma palavra para a seguinte apenas muda um bit. Estas palavras dizem-se adjacentes.

Duas palavras dizem-se adjacentes se representam algarismos decimais consecutivos e só diferem num bit [2].

Um código diz-se contínuo se for formado só por palavras adjacentes [6].

Um código contínuo diz-se cíclico se a última palavra for adjacente da primeira. [6]

Como se pode verificar o código Johnson é cíclico.

1.4.1 Adição em BCD

A adição de dois números representados em código BCD não tem dificuldades desde que as comas dos grupos de 4 bites não exceda 9. Apresenta-se um exemplo na figura 1.15 [7].

BCD (8421) ADDITION

Example:	
decimal	BCD
427	0100 0010 0111
+ 131	+ 0001 0011 0001
558	0101 0101 1000

Figura 1.15 Soma BCD sem necessidade de correcção

Contudo se o leitor somar 6+4 obtém um padrão de bites que não pertence ao código. O valor binário estará correto $1010_2 = 10$ mas o resultado deveria ser $1\ 0000_{BCD}$. Ora como este último valor em binário vale 16 verifica-se que deverá existir uma soma de correcção somando 6 ao

resultado em binário. O leitor pode verificar que o mesmo acontece quando soma números altos, por exemplo 9+8, que geram um transporte depois do quarto bite. De novo a soma de correcção obtém-se adicionando 6. Na figura 1.16 ilustra-se uma operação com somas correctivas [7].

BCD ADDITION: +6 CORRECTION

$$\begin{array}{r} 789 \\ + 213 \\ \hline 1002 \end{array} \quad \begin{array}{r} 0111 \ 1000 \ 1001 \\ + 0010 \ 0001 \ 0011 \\ \hline 1001 \ 1001 \ 1100 \\ + \ 0110 \quad +6 \text{ correction} \\ \hline 1001 \ 1010 \ 0010 \\ + \ 0110 \quad +6 \text{ correction} \\ \hline 1010 \ 0000 \ 0010 \\ + \ 0110 \quad +6 \text{ correction} \\ \hline 1 \ 0000 \ 0000 \ 0010 \end{array}$$

Figura 1.15 Soma BCD com necessidade de correcção

Apresenta-se outro exemplo em que há um transporte depois de um grupo de 4 bites e respectiva correcção.

$$\begin{array}{r} 1 \\ 0001 \ 0010 \ 1001 \\ + 0010 \ 0100 \ 0111 \\ \hline 0011 \ 0111 \ 0000 \\ + \ 0110 \\ \hline 0011 \ 0111 \ 0110 \end{array}$$

Figura 1.16 Soma BCD com transporte gerado por um grupo de 4 bites

Para a subtracção em BCD considera-se o complemento do subtrativo, na base 10, e depois codificam-se os algarismos, mantendo as correcções da soma atrás enumeradas.

1.4.2 Adição em BCD com excesso de 3

Se o leitor somar 0+0 em código BCD com excesso de 3 verifica que o resultado tem um excesso de 3 unidades para o valor pretendido. Por outro lado se somar 9+9 o resultado exhibe um erro, para o padrão correto do mesmo código, de -3 unidades. Assim, o algoritmo de correcção, chamado de Stibitz [7], enuncia-se de seguinte forma: Se houver transporte para fora de um grupo de 4 bites deve somar-se 3 (0011), se não houver transporte deve subtrair-se 3, somando o complemento (1100 ou 1101). A grande vantagem é que a soma de correcção não gera transporte para fora do grupo de 4 bites, pelo que as correcções dos algarismos da soma binária podem ser feitas em simultâneo [7].

Exemplo Num registo de 16 bites, sendo 4 de sinal, calcule 937-629, numa representação em complemento para a base-1. O cálculo ilustra-se na figura 1.17.

$$\begin{array}{r}
 629 \quad 0629 \quad 0011 \ 1001 \ 0101 \ 1100 \\
 -629 \quad 9370 \quad 1100 \ 0110 \ 1010 \ 0011 \\
 937 \quad 0937 \quad 0011 \ 1100 \ 0110 \ 1010 \\
 \hline
 1 \ 0000 \ 0011 \ 0000 \ 1101 \quad 1 \\
 \hline
 0000 \ 0011 \ 0000 \ 1110 \\
 0011 \ 0011 \ 0011 \ 1100 \\
 \hline
 0011 \ 0110 \ 0011 \ 1010 \quad 1 \\
 \hline
 0011 \ 0110 \ 0011 \ 1011
 \end{array}$$

Figura 1.17 Cálculo de 937-629 em BCD xc3 pelo algoritmo de Stibitz.

Note que o complemento pode ser calculado em decimal e representar os algarismos em código de seguida, mas também pode ser feito por troca de símbolos a partir da codificação de 629, já que o código é autocomplementável. Os traços a vermelho indicam transportes finais a somar e o traço horizontal com espaços indica que a soma termina a cada grupo de 4 bites.

Referências Bibliográficas do capítulo 1

Complementar

[1] Giancarlo Masini, A Matemática: O Romance dos Números, edição do Círculo de Leitores.

Fundamental

[2] Guilherme Arroz e outros, Arquitetura de Computadores: Dos sistemas digitais aos microprocessadores, IST PRESS, 2ª Edição 2009.

[3] - Binary Coded Decimal, Wikipédia em 16/3/2010, ver 'Subtraction with BCD'

[4] - Sandige, R., Modern Digital Design, McGraw-Hill, 1990.

[5] - Wikipédia em 22 de Junho de 2015.

[6] - Borges R. Apontamentos de Sistemas Digitais, publicação interna da Universidade de Aveiro.

[7] - Podor, Balint, Digital Technics I, Obuda University, Microelectronics and technology Institute, publicação na net, 2012.

Capítulo 2 Álgebra de Boole

A aritmética binária, tal como os códigos binários, baseiam-se exclusivamente no uso dos algarismos zero e um para representar qualquer número real. Por outro lado os circuitos digitais usam uma faixa de tensão pré-definida para sinalizarem o símbolo zero e uma outra faixa, não sobreponível com a anterior, para o símbolo um. O projecto e concepção de sistemas digitais complexos baseia-se numa ferramenta teórica designada por álgebra de Boole, que permite exprimir matematicamente uma dada função booleana e simplifica-la.

Para exemplificar variáveis e funções binárias, considere-se um semáforo de duas cores, em que o verde é representado pelo símbolo zero e o vermelho pelo símbolo um. A acção de um condutor humano em frente do semáforo é binária, ou seja, este pára no semáforo ou arranca. Associemos a primeira situação, a paragem, ao símbolo zero e a segunda ao símbolo um. A função do comportamento humano pode agora ser representada por uma tabela de verdade:

Variável 'x'	Variável 'y'
Verde	arranca
Vermelho	pára

Ou, usando um formalismo matemático:

Variável 'x'	Variável 'y'
0	1
1	0

A função $y=f(x)$ corresponde à negação, ou complementação, ou inversão. Representa-se simplesmente por:

$$y = \bar{x}$$

Para funções de uma variável apenas existem quatro possibilidades [1]:

$Y=0$ independentemente de x ser 0 ou 1 - Constante zero

$Y=1$ independentemente de x ser 0 ou 1 - Constante um

$Y=x$ - Função identidade

$y = \bar{x}$ - Função complementar

O facto de numa função binária se poderem considerar todas as possibilidades, permite demonstrar teoremas por indução completa. Para ilustrar este conceito apresenta-se o teorema da dupla negação:

$$\bar{\bar{x}} = x$$

x	$\bar{x}$	$\bar{\bar{x}}$
0	1	0
1	0	1

Bases de numeração, códigos e Álgebra de Boole

Funções de duas variáveis

As funções de duas variáveis são também em número limitado, neste caso dezasseis. Apresentam-se as mais importantes.

A função conjunção ou produto lógico - Considerem-se as duas frases:

Miguel tem vinte anos. a

Maria, sua irmã, tem dez anos. b

Qualquer uma das frases pode ser verdadeira ou falsa. Considere-se agora a conjunção das duas frases, Miguel tem vinte anos e Maria, sua irmã, tem dez anos.

Esta frase composta pela conjunção das primeiras só é verdadeira se e só se as afirmações a e b forem ambas verdadeiras. Os vários casos podem ser apresentados numa tabela de verdade:

a	b	$a \cap b$
Falso	Falso	Falso
Falso	Verdadeiro	Falso
Verdadeiro	Falso	Falso
Verdadeiro	Verdadeiro	Verdadeiro

Considerando que Falso é associado ao zero lógico e Verdadeiro ao um tem-se:

a	b	$a \cap b$
0	0	0
0	1	0
1	0	0
1	1	1

Se interpretarmos a tabela numericamente, verifica-se facilmente que a função em estudo é um produto, pelo que é designada em sistemas digitais por produto lógico, e representadas como:
 $y=ab$

A função disjunção exclusiva ou soma lógica exclusiva.

Considerem-se duas frases:

Logo vou ao cinema a

Logo vou ao teatro b

À frase composta 'Logo vou ao cinema ou logo vou ao teatro' chama-se a disjunção exclusiva. A sua tabela de verdade é:

a	b	$a \cup b$
Falso	Falso	Falso
Falso	Verdadeiro	Verdadeiro
Verdadeiro	Falso	Verdadeiro
Verdadeiro	Verdadeiro	Falso

Utilizando zeros e uns tem-se:

Bases de numeração, códigos e Álgebra de Boole

a	b	$a \cup b$
0	0	0
0	1	1
1	0	1
1	1	0

Note-se que se ambas as proposições, ‘Logo vou ao cinema’ e ‘Logo vou ao teatro’ forem verdadeiras, por absurdo que pareça, a disjunção é falsa, resultando daqui o nome de disjunção exclusiva. Note-se que uma interpretação numérica da tabela de verdade mostra que a terceira linha é a soma das duas primeiras, esquecendo o transporte gerado na última linha, vindo daqui o nome de soma lógica. Simbolicamente a função representa-se assim:

$$y = a \oplus b$$

A função disjunção inclusiva ou soma lógica inclusiva.

Considerem-se as duas frases:

João triunfou na vida porque é esperto. a

João triunfou na vida porque teve sorte. b

À frase composta ‘João triunfou na vida porque é esperto’ ou ‘João triunfou na vida porque teve sorte’ chama-se disjunção inclusiva, ou simplesmente soma lógica. A tabela de verdade é:

a	b	$a \cup b$
Falso	Falso	Falso
Falso	Verdadeiro	Verdadeiro
Verdadeiro	Falso	Verdadeiro
Verdadeiro	Verdadeiro	Verdadeiro

Utilizando zeros e uns tem-se:

a	b	$a \cup b$
0	0	0
0	1	1
1	0	1
1	1	1

Registe-se que ambas as proposições a e b podem ser verdadeiras, que a disjunção inclusiva se mantém verdadeira, ou seja João pode ao mesmo tempo ser esperto e ter sorte.

A soma lógica representa-se simplesmente por:

$$y = a + b$$

Atenção: Doravante quando se falar apenas em soma lógica referimo-nos sempre à soma inclusiva. Qualquer referência à soma exclusiva exige o uso desta palavra.

Bases de numeração, códigos e Álgebra de Boole

Fundamentação rigorosa da álgebra de Boole aplicada a variáveis binárias.

A fundamentação rigorosa é feita recorrendo a axiomas que são considerados verdades evidentes. Considere-se um conjunto não vazio B com os seguintes elementos:

$$B = \{0,1\}$$

Considerem-se duas operações designadas por soma lógica e produto lógico, tendo-se os seguintes axiomas que os alunos podem validar consultando as tabelas acima referidas.

Axioma 1 - As duas operações, soma e produto lógico, são fechadas em B , isto é se a e b pertencem a B então:

$$(a + b) \in B$$

$$(ab) \in B$$

Axioma 2 - As duas operações são comutativas, isto é, o resultado não depende da ordem dos operandos:

$$(a+b)=(b+a)$$

$$ab=ba$$

Axioma 3 - Existem elementos neutros para as duas operações.

$$a+0=a$$

$$b \cdot 1 = b$$

Axioma 4 - Distributividade das duas operações uma em relação à outra.

$$a + (b \cdot c) = (a + b) \cdot (a + c)$$

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

A primeira parte do axioma 4 não parece verdade. Vamos verificar a validade por indução completa, ou seja, explicitando todos os possíveis valores das variáveis e confirmando, ou não, se a igualdade é verdadeira

a	b	c	bc	a+bc	a+b	(a+c)	(a+b)(a+c)
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	0	0	1	1	1	1
1	1	1	1	1	1	1	1

Como se verifica a quinta coluna é igual à última, pelo que está verificada a igualdade. Um exercício semelhante pode comprovar a distributividade do produto relativamente à soma, algo que nos é familiar.

Bases de numeração, códigos e Álgebra de Boole

Axioma 5 - Complementação. Para todo o elemento a de B , existe um elemento $\bar{a}$ de B tal que:

$$a + \bar{a} = 1$$

$$a \cdot \bar{a} = 0$$

Com base nestes axiomas deduzem-se alguns teoremas importantes.

Teorema da idempotência $a+a=a$

Demonstração.

$$a+0=a \quad \text{por Axioma 3, (A3).}$$

$$a + a\bar{a} = a \quad \text{por A5.}$$

$$(a + a) \cdot (a + \bar{a}) = a \quad \text{por A4.}$$

$$(a + a) \cdot 1 = a \quad \text{por A5}$$

$$a + a = a \quad \text{por A3}$$

Quod est demonstratum. (Esta é a parte que vocês não percebem - Latim)

Em álgebra de Boole existe um conceito extremamente importante que é a dualidade. Os axiomas definiram pares de igualdades, obtendo-se uma delas a partir da outra, bastando para tal mudar o sinal operador e trocar zero por um e vice-versa.

Assim por dualidade também haverá um teorema da idempotência para o produto lógico, que será $a \cdot a = a$

De facto pode repetir-se a demonstração usando a igualdade dual para cada axioma. Assim tem-se:

$$a \cdot 1 = a$$

$$a \cdot (a + \bar{a}) = a$$

$$aa + (a\bar{a}) = a$$

$$aa + 0 = a$$

·Como queríamos demonstrar (Agora já percebem o latim!)

Doravante não se demonstra o teorema dual. Repare-se que nesta álgebra não há operações inversas.

Apresentam-se alguns teoremas importantes cuja dedução, por axiomas ou por indução completa, podem ser demonstrados pelos estudantes.

Teorema dos elementos absorventes: $a+1=1$

Dual

$$a \cdot 0 = 0$$

Teorema da absorção: $a(a+b)=a$

Dual ?

Associatividade: $a+(b+c)=(a+b)+c$

Sugere-se a prova por indução completa.

Leis de Morgan: $\overline{a+b} = \bar{a} \cdot \bar{b}$

$$\overline{a \cdot b} = \bar{a} + \bar{b}$$

Dedução da primeira lei:

Seja $p=a+b$ e $q= \bar{a} \cdot \bar{b}$

Calcule-se pq e $p+q$

$$pq = (a + b)(\bar{a} \cdot \bar{b})$$

$$=(a\bar{a} + b\bar{a})\bar{b}$$

$$= (0 + b\bar{a})\bar{b}$$

$$= (b\bar{a})\bar{b}$$

$$= (b\bar{b})\bar{a}$$

$$= 0 \cdot \bar{a}$$

$$=0$$

$$p + q = (a + b) + \bar{a} \cdot \bar{b}$$

$$= a + (b + \bar{a} \cdot \bar{b})$$

$$= a + (b + \bar{a})(b + \bar{b})$$

$$= a + (b + \bar{a}) \cdot 1$$

$$= (a + \bar{a}) + b$$

$$= 1+b$$

$$=1$$

Assim $pq=0$ e $p+q=1$. Logo pelo axioma 5, da complementaridade, p e q têm que ser complementares:

$$\bar{p} = q$$

Ou finalmente

Bases de numeração, códigos e Álgebra de Boole

$$\overline{a + b} = \bar{a} \cdot \bar{b}$$

Exemplo de uma função booleana:

$$z = (a \cdot b) + (\bar{a} \cdot \bar{b})$$

A função pode escrever-se sem os parêntesis:

$$z = ab + \bar{a} \cdot \bar{b}$$

Esta função pode ser representada por uma tabela de verdade, atribuindo todos os possíveis valores a a e b.

a	b	ab	$\bar{a} \cdot \bar{b}$	z
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

É equivalente expressar a função algebricamente ou por tabela de verdade.

Símbolos lógicos usados nos circuitos digitais que executam as funções lógicas.

AND



OR



NAND



NOR



Inversor



XOR



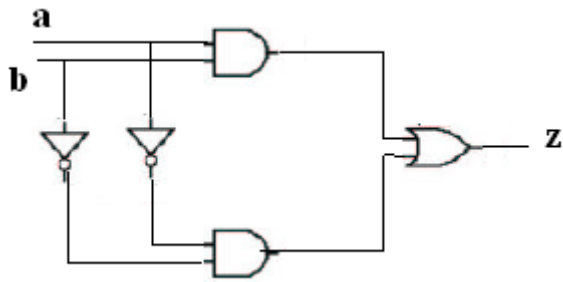
Figura 1 - Símbolos Lógicos [1]

Traduzindo as siglas tem-se: AND = E OR = OU
XOR=OU-EX

NAND = NE

NOR = NOU

O circuito que realizaria a função do exemplo anterior seria:



Obtenção de uma função booleana partindo da tabela de verdade da função.

Introduz-se um conceito importante:

Termo mínimo é uma função booleana só constituída por zeros, excepto para uma dada combinação de entradas.

Ilustra-se o conceito com a tabela de verdade do produto lógico:

a	b	ab
0	0	0
0	1	0
1	0	0
1	1	1

O produto lógico é uma função booleana que é um termo mínimo. Apresentam-se outros exemplos:

a	b	f
0	0	0
0	1	0
1	0	1
1	1	0

Neste caso tem-se $f = a\bar{b}$. Repare-se que o termo mínimo não pode ser uma soma de variáveis, senão a função teria o valor 1 duas vezes. Usa-se a variável que está a 1 multiplicada pelo complemento da que está a zero.

Terceiro exemplo:

a	b	f
0	0	0
0	1	1
1	0	0
1	1	0

$$f = \bar{a}b$$

Quarto exemplo:

a	b	f
0	0	1
0	1	0
1	0	0
1	1	0

$$f = \bar{a} \cdot \bar{b}$$

Quinto exemplo: Calcular a função booleana que representa a operação ou-exclusivo.

Repete-se a tabela de verdade do ou-exclusivo:

a	b	$y = a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

Repare-se que a função pode ser considerada como o ‘ou’ do segundo e terceiro exemplo de termos mínimos da página anterior, pelo que se tem:

$$y = \bar{a}b + a\bar{b}$$

A primeira parcela é responsável pelo 1 da combinação 01 e a segunda parcela gera o 1 para a combinação 10. Fora destas combinações as parcelas introduzem zero na soma, que é o elemento neutro desta. São precisas cinco portas lógicas para executar esta função. Verifique!

Uma maneira alternativa de representar a função é indicar em índice o valor decimal correspondente à combinação binária responsável pelo termo mínimo. Neste caso seria:

$$y = m_1 + m_2$$

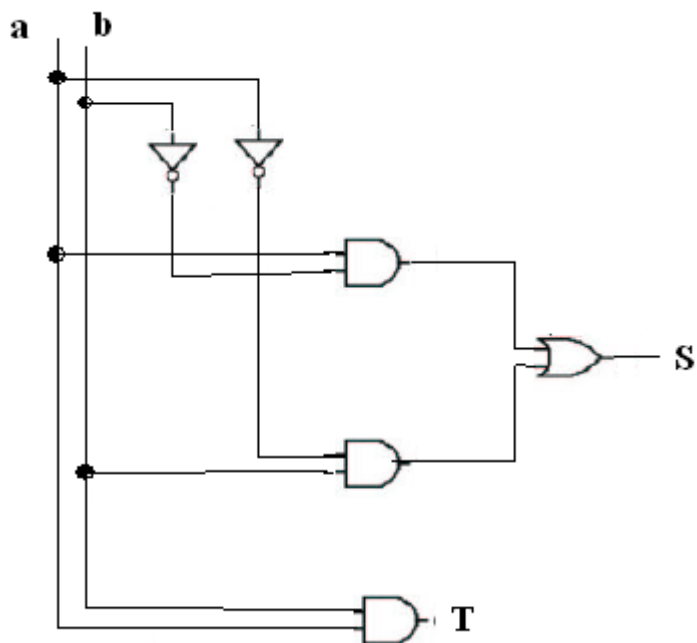
A decomposição de uma função na soma de termos mínimos chama-se a primeira forma canónica, ou forma canónica soma de produtos.

Exemplo: Projectar um semi-somador de um bit. É fácil verificar que a soma de a com b produz um resultado de dois bits. Logo são precisas duas funções uma para a soma, outra para o transporte. É evidente que:

$$S = a \oplus b$$

$$T = ab$$

Pelo que o desenho do circuito é agora fácil.



Circuito semi-somador de um bit - Esquema padrão para colocação de portas